CS430/910: Foundations of Data Analytics

Tools for Data Analysis | Dr Ayse Sunar

Objectives

- Introduce a broad collection of tools used in data analytics
- Outline the capabilities and uses of each tool
  - Provide examples of tool usage
- Allow you to select the appropriate tools to work with
  - Based on your preferences, e.g. GUI or command line
- Very quick introduction to each tool
  - More information on the web, in the library

The Philosophy of Tool Choice

The right tool for the right job

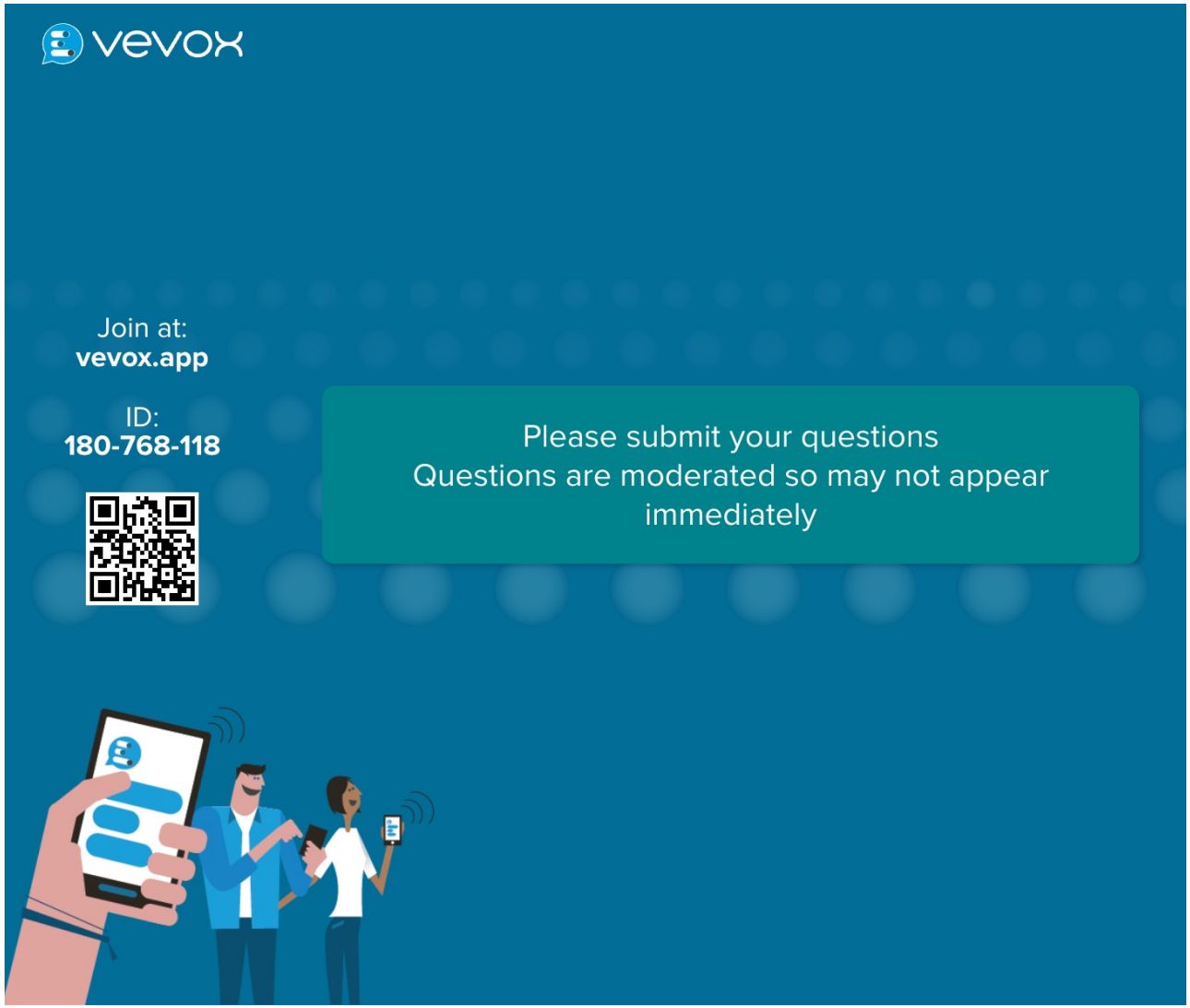
- A huge array of tools is available, with overlapping functionality
- A good data analyst knows a good selection of tools
  - Can pick the right one for the job
- Many people know only one or two (often unsuitable) tools
  - Hence, much of the world's analytics is performed in spreadsheets

The Philosophy of Tool Choice

- Knowing that a tool exists and what it can do is often enough
  - Can decide if learning to use it is time-effective
  - Will introduce some options here
  - Will **not** give formal training
- No single answer to tool selection
  - Often a matter of personal choice

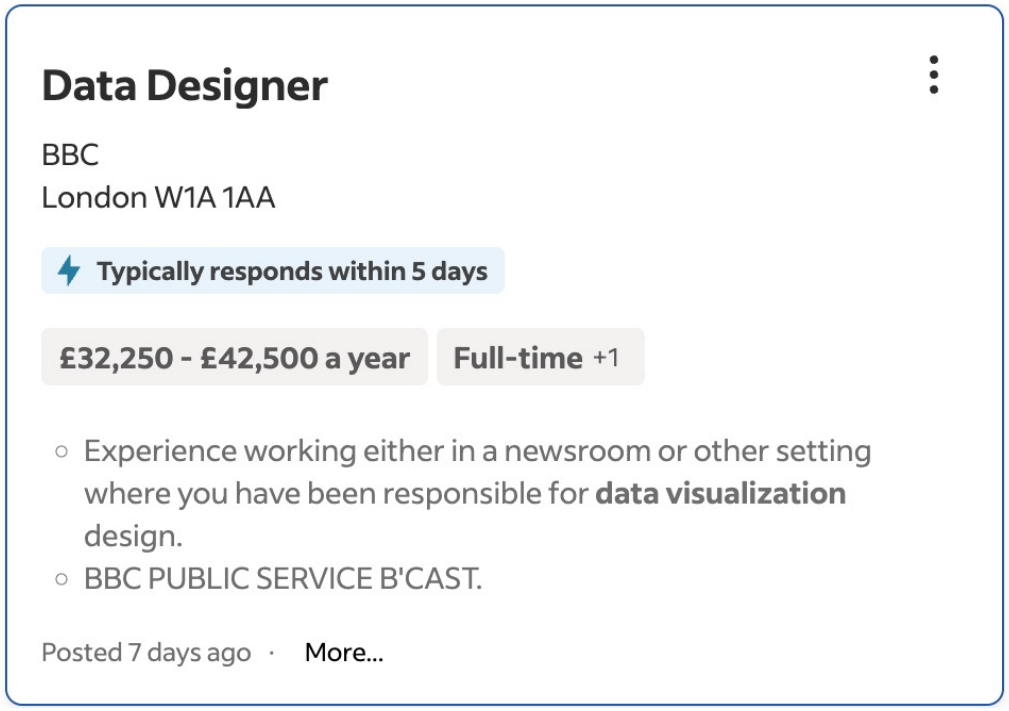
Let’s find out our trend in tools!

Vevox.app



Rough Categorisation of Tools

- Data Manipulation
- Data Visualization
- Data Analysis



Data Designer

BBC
London W1A 1AA

Typically responds within 5 days

£32,250 - £42,500 a year | Full-time +1

- Experience working either in a newsroom or other setting where you have been responsible for **data visualization** design.
- BBC PUBLIC SERVICE B'CAST.

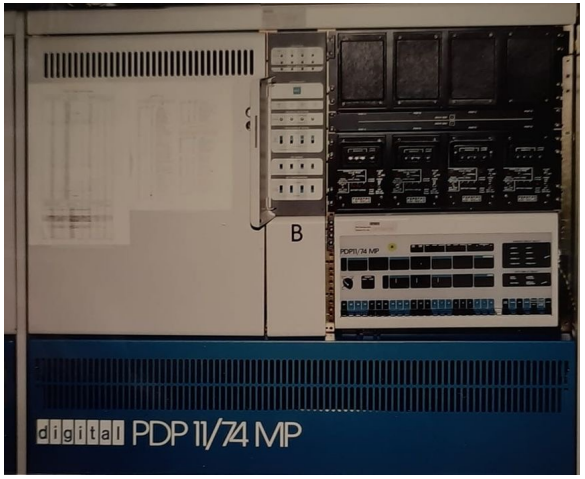
Posted 7 days ago · More...

Unix Tools

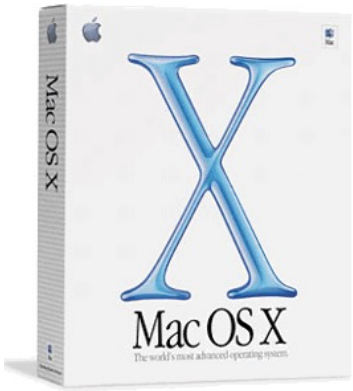
- “Unix tools” covers many simple tools developed as part of the Unix operating system
  - They manipulate data files represented as lines of text: flat files, comma separated value (CSV) files
  - Allow simple analysis and data preparation
  - Widely available in Linux, MacOS, Windows
- “I use all these nearly every day. The best part is, once you know they exist, these tools are available on every unix machine you will ever use. Nothing else (except maybe perl) is as universal – you don’t have to worry about versions or anything. Being comfortable with these tools means you can get work done anywhere – any EC2 instance you boot up will have them, as will any unix server you ssh into.”

Unix History in a Nutshell

- Developed at AT&T Bell Labs in late 1960's for PDP11 (hardware)
  - Made available in mid-1970's
  - Developed and sold by AT&T in the 1980's
  - Commercial variants emerged: Solaris, SCO…
- Standardised via POSIX in 1989
  - POSIX: Portable Operating System Interface based on Unix
- GNU foundation launched free implementations in 1980s
  - Linux started in 1991 as a free POSIX-compliant OS kernel
  - Many Linux distributions available: Ubuntu, Fedora, Debian…

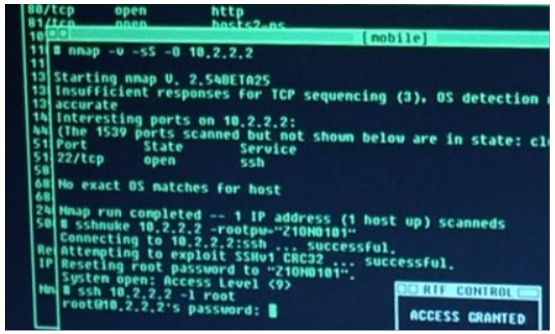


Tool Availability



- Available on any Unix machine
- Available on any Linux machine
  - Such as those in DCS, e.g. grace, stone and cobra
- Available on any modern Mac
  - Open the ‘console’ and type away
- On Windows:
  - Various ports of individual tools or collections of tools
  - Cygwin, open source port of many linux tools to Windows (http://cygwin.com/install.html)

Command Line Tools



nmap.org/movies/

- These are command line tools – no fancy GUI
- Each tool performs a single simple function
  - Additional functionality has crept in over time
  - Now some are more like a swiss army knife
- Can be combined via scripts, piping
- Information available on each tool:
  - Via ‘man’ command: e.g. man cat
  - Via program itself: sort –help
  - Via the web: many instructions/examples online
- Short course on unix tools from Cambridge:
  - http://www.cl.cam.ac.uk/teaching/1213/UnixTools/materials.html

Cheat sheets

- A summary collection of most needed commands

Unix/Linux Command Reference

FOSSwire.com

File Commands

ls – directory listing
ls -al – formatted listing with hidden files
cd *dir* - change directory to *dir*
cd – change to home
pwd – show current directory
mkdir *dir* – create a directory *dir*
rm *file* – delete *file*
rm -r *dir* – delete directory *dir*
rm -f *file* – force remove *file*
rm -rf *dir* – force remove directory *dir* *
cp *file1 file2* – copy *file1* to *file2*
cp -r *dir1 dir2* – copy *dir1* to *dir2*; create *dir2* if it doesn't exist
mv *file1 file2* – rename or move *file1* to *file2* if *file2* is an existing directory, moves *file1* into directory *file2*
ln -s *file link* – create symbolic link *link* to *file*
touch *file* – create or update *file*
cat > *file* – places standard input into *file*
more *file* – output the contents of *file*
head *file* – output the first 10 lines of *file*
tail *file* – output the last 10 lines of *file*
tail -f *file* – output the contents of *file* as it grows, starting with the last 10 lines

Process Management

ps – display your currently active processes
top – display all running processes
kill *pid* – kill process id *pid*
killall *proc* – kill all processes named *proc* *
bg – lists stopped or background jobs; resume a stopped job in the background
fg – brings the most recent job to foreground
fg *n* – brings job *n* to the foreground

File Permissions

chmod *octal file* – change the permissions of *file* to *octal*, which can be found separately for user, group, and world by adding:

- 4 – read (r)
- 2 – write (w)
- 1 – execute (x)

Examples:
chmod 777 – read, write, execute for all
chmod 755 – rwx for owner, rx for group and world
For more options, see **man chmod**.

SSH

ssh *user@host* – connect to *host* as *user*
ssh -p *port user@host* – connect to *host* on port *port* as *user*
ssh-copy-id *user@host* – add your key to *host* for *user* to enable a keyed or passwordless login

Searching

grep *pattern files* – search for *pattern* in *files*
grep -r *pattern dir* – search recursively for *pattern* in *dir*
command* | grep *pattern – search for *pattern* in the output of *command*
locate *file* – find all instances of *file*

System Info

date – show the current date and time
cal – show this month's calendar
uptime – show current uptime
w – display who is online
whoami – who you are logged in as
finger *user* – display information about *user*
uname -a – show kernel information
cat /proc/cpuinfo – cpu information
cat /proc/meminfo – memory information
man *command* – show the manual for *command*
df – show disk usage
du – show directory space usage
free – show memory and swap usage
whereis *app* – show possible locations of *app*
which *app* – show which *app* will be run by default

Compression

tar cf *file.tar files* – create a tar named *file.tar* containing *files*
tar xf *file.tar* – extract the files from *file.tar*
tar czf *file.tar.gz files* – create a tar with Gzip compression
tar xzf *file.tar.gz* – extract a tar using Gzip
tar cjf *file.tar.bz2* – create a tar with Bzip2 compression
tar xjf *file.tar.bz2* – extract a tar using Bzip2
gzip *file* – compresses *file* and renames it to *file.gz*
gzip -d *file.gz* – decompresses *file.gz* back to *file*

Network

ping *host* – ping *host* and output results
whois *domain* – get whois information for *domain*
dig *domain* – get DNS information for *domain*
dig -x *host* – reverse lookup *host*
wget *file* – download *file*
wget -c *file* – continue a stopped download

Installation

Install from source:
./configure
make
make install
dpkg -i *pkg.deb* – install a package (Debian)
rpm -Uvh *pkg.rpm* – install a package (RPM)

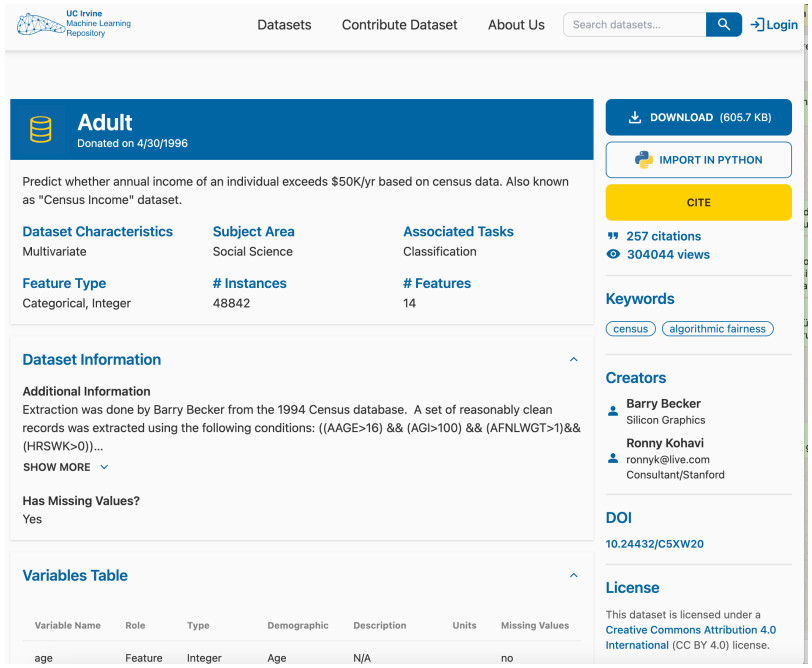
Shortcuts

Ctrl+C – halts the current command
Ctrl+Z – stops the current command, resume with **fg** in the foreground or **bg** in the background
Ctrl+D – log out of current session, similar to **exit**
Ctrl+W – erases one word in the current line
Ctrl+U – erases the whole line
Ctrl+R – type to bring up a recent command
!! - repeats the last command
exit – log out of current session

* use with extreme caution.

cc BY SA

Example Data Set



- Show examples using the “adult census data”
  - http://archive.ics.uci.edu/dataset/2/adult/
  - Click on Download
- ~32K individuals, one per line
  - Age, Sex, Employment Type, Years of Education…
- Widely studied in Machine Learning community
  - Prediction Task: Is income > 50K?

Standard Input, Output and Piping

- Unix commands can read and write files
  - Special case: standard input (stdin) and standard output (stdout)
  - By default, a command reads from stdin, writes to stdout
- Some commonly used tools are *wc* and *cat*
  - *wc* does a simple wordcount
  - cat reads a file, writes it to stdout
  - Pipe '|' connects the stdout of one command to stdin of next

- Examples:
  - cat adult.data | wc
    - Output: 32562 488415 3974305 [lines words characters]
  - cat adult.data| wc | wc
    - Output: 1 3 24

Redirection

wc options:
-l / -w / -c : print number of lines / words / characters

- Can use < to redirect a file to stdin, and > to redirect stdout
  - >> appends to an existing file
- Examples:
  - wc < adult.data
    - 32562 488415 3974305
  - wc < adult.data > wordcount
- cat wordcount
    - 32562 488415 3974305
  - cat adult.data | wc >> wordcount

Basic Commands

- *ls*: list files in a directory
  - ls adult

    adult.data adult.names adult.test
- Options to commands are often single letters preceded by -
  - ls –l adult

    total 5852

    -rwx------ 1 grahamc dcsstaff 3974305 Oct 8 18:03 adult.data

    -rwx------ 1 grahamc dcsstaff   5229 Oct 8 18:03 adult.names

    -rwx------ 1 grahamc dcsstaff 2003153 Oct 8 18:04 adult.test

Basic Commands

- ls –la public_html

```
total 5860
drwx------  2 grahamc dcsstaff    4096 Oct  8 18:04 .
drwx------ 39 grahamc dcsstaff    4096 Oct  8 18:04 ..
-rwx------  1 grahamc dcsstaff 3974305 Oct  8 18:03 adult.data
-rwx------  1 grahamc dcsstaff    5229 Oct  8 18:03 adult.names
-rwx------  1 grahamc dcsstaff 2003153 Oct  8 18:04 adult.test
```

Viewing Files: cat, head, tail

- *cat* file shows contents of file
- *head* shows first few lines of a file
  - head adult.data

    39, State-gov, 77516, Bachelors, 13, Never-married,

    50, Self-emp-not-inc, 83311, Bachelors, 13, Married-civ-spouse,

    38, Private, 215646, HS-grad, 9, Divorced,

    53, Private, 234721, 11th, 7, Married-civ-spouse, Handlers-cleaners,

    28, Private, 338409, Bachelors, 13, Married-civ-spouse, Prof-specialty,

    37, Private, 284582, Masters, 14, Married-civ-spouse, Exec-managerial,

    49, Private, 160187, 9th, 5, Married-spouse-absent, Other-service,

Viewing Files: cat, head, tail

- *Tail* shows last few lines of a file
  - Tail –n 5 adult.data

40, Private, 154374, HS-grad, 9, Married-civ-spouse, Machine-op-inspct,

58, Private, 151910, HS-grad, 9, Widowed, Adm-clerical, Unmarried, White,

22, Private, 201490, HS-grad, 9, Never-married, Adm-clerical, Own-child,

52, Self-emp-inc, 287927, HS-grad, 9, Married-civ-spouse, Exec-managerial,

Viewing Files: more, less

- *more* lets you page through a file
  - Page down/space to advance
- *less* is a faster replacement
- Can *page up* to go back
- *Q* to quit

The sort command

- *sort*: sorts the input
- Default: sort lines by alphabetic order
- sort adult.data
- Configurable
  - -r: reverse sort
  - -n: numeric sort
  - -k: column on which to sort (assume space separates fields)
    - sort adult.data –k5 | less
    - sort adult.data –n –k5 | less
  - -f: ignore (upper/lower) case

The cut command

- *cut*: select certain columns from the file
  - Default: assume tab separates columns
  - -f: specify which fields to select
  - -c: specify which character positions in each line to select
    - cut –c1-9 adult.data | head
    - cut –c1,3,5,7,9 adult.data | head
- -d: specify the field delimiter
  - cut –f1 adult.data | head
  - cut –f1,2 –d, adult.data | head
  - cut –f1,3,5,7,9 –d, adult.data | head

The uniq command

- *uniq*: omit (or report) repeated lines
  - cut –f1 –d, adult.data | uniq | head
  - cut –f1 –d, adult.data | sort –n | uniq | head
- Count the number of occurrences with -c
  - cut –f1 –d, adult.data | sort –n | uniq –c | head
  - cut –f1 –d, adult.data | sort –n | uniq –c | sort –rn | head

The grep command

- *grep*: search for lines that match some text
  - grep Masters adult.data | head
  - grep Masters adult.data | wc –l
- -i: ignore case
- -v: invert behaviour, select non-matching lines
- -An, -Bn: print n lines of context appearing After / Before the match
  - grep –A1 –B2 Hungary adult.data | less
- Can handle **regular expressions** for flexible matching:
  - grep “Married.*England” adult.data | less
  - grep ^90 adult.data | less
  - (More info on next slide)

grep + Regular Expressions

patterns used for searching and manipulating strings in text processing

- Grep's regular expression syntax:
  - ^ : start of line
  - $: end of line
  - \ : "escape" next character: \$ to match a $ sign
  - [abc] : match any character of abc
  - [a-z] : match any character in range a to z
  - . (dot) : match any character
  - * : match 0 or more occurrences of preceding expression
  - \{n\} : match n instances of preceding expression

- Example: grep "\(21\)\{2\}" adult.data
- egrep for "extended" regular expressions:
  - egrep "England|Mexico" adult.data | head

The sed command

- *sed*: stream editor
  - Most commonly used to substitute some text for others
  - sed ‘s/expression/replacement/g’
    - sed ‘s/Private/Secret/g’ adult.data | head
    - sed ‘s/, /\t/g’ adult.data | head
    - sed ‘s/, /\n/g’ adult.data | head

The join command

- *join*: do a database-style join on two sorted text files
  - -1 n -2 m: try to match n'th field of first file with m'th field of second
  - Output all combinations of matches
  - e.g. join list of people + postcodes with average income in postcode
- Example:
  - grep United-States -v adult.data | head -n 20 | cut -f 4,14 -d, | sort –k2 > adult.join1

    grep United-States -v adult.data | head -n 20 | cut -f 1,14 -d, | sort –k2 > adult.join2

    join -1 2 -2 2 adult.join1 adult.join2

Acknowledgements

- The slides use ideas/images from
  - G. Cormode [Warwick, CS910]
  - F. Ciucu [Warwick, CS430/CS910]